

Trajector: Pathwise Adjoint Correction of Step-Cached Diffusion Transformers

Ruize Xia

High School Student

Nanjing Foreign Language School

Nanjing, China

xia.ruize0911@gmail.com

Abstract—Step caching accelerates diffusion transformers by reusing block residuals across solver steps, at a cost in image quality. A learned corrector can predict the reuse error; the natural way to train it is residual regression. We show that this objective is misaligned with deployed quality: an anchor refreshes the cache but not the latent, so cache error integrates along the trajectory while residual error is pointwise. Retraining the residual corrector on more data to convergence lowers residual error by 25–30% yet changes SSIM by only -0.002 to $+0.012$. We therefore fine-tune the same corrector with Trajector (*pathwise adjoint correction*), which minimises latent trajectory deviation by differentiating through the cached rollout. On held-out prompts, Trajector improves SSIM over the residual-trained corrector at every injection granularity we tested ($K=1$ to 28 correction sites per step, paired $t=4.5$ to 10.0 , $n=72$) and LPIPS at every $K \geq 2$. At a fixed step count on PixArt- Σ -XL-2-512 it reaches SSIM 0.532 against the exact output at $3.39\times$ speedup, $+0.098$ above a reproduction of TaylorSeer ($t=12.8$) and $+0.053$ above one of Block Caching ($t=9.5$), at 8% and 3% less speedup. Under ImageReward, only state-dependent learned correction improves on verbatim caching, and among these the residual objective scores higher than Trajector.

Index Terms—diffusion transformers, feature caching, pathwise objectives, inference acceleration

1. Introduction

Sampling a diffusion transformer [1] costs a full forward pass per solver step. Step caching exploits how slowly block outputs change: an *anchor* step, taken every i steps, stores each block’s residual $R = \text{block}(h) - h$ for input h , and the next $i - 1$ *reuse* steps skip the block and add the stored residual instead [2], [3], [4]. The method needs no training and is fast, but it visibly degrades the image.

Besides choosing *when* to reuse [5], [6], two remedies exist. Training-free extrapolation forecasts the residual from its own history [7]. Learned correction trains a model, from a timestep-conditioned scale and shift [8] to a network that reads the current state (ours), to predict the reuse error

$$\Delta R = R_{\text{true}} - R_{\text{anchor}}, \quad (1)$$

where R_{true} is the residual the block would produce on the current hidden state and R_{anchor} is the one stored at the most recent anchor. ΔR is exactly the error verbatim reuse commits. This paper studies the second family. Its default training signal is supervised regression on (1), and every later design choice (architecture, checkpoint selection, and how much correction to inject) is then made in residual space.

We show that this signal is misaligned with deployed quality, and that a cheap second training stage recovers much of what it misses. Concretely, we contribute:

- 1) **A diagnosis of the residual objective.** The cached trajectory’s deviation from the exact one climbs almost monotonically (0.0089 at step 1 to 0.5856 at step 19): an anchor refreshes the cache, not the latent. Retraining with matched data lowers residual error by 25–30% but changes SSIM by only -0.002 to $+0.012$.
- 2) **Trajector.** The corrector can act once per step or at each of K groups of blocks. Keeping the architecture, schedule and deployment rule, we fine-tune the corrector by differentiating through the cached rollout. A cached block never runs, so the graph over one reuse step is K corrector forwards rather than the 28 blocks of the 0.6B model. It improves SSIM at every K , with a $2\text{--}3\times$ larger margin at $K \geq 4$, where residual training degrades most.
- 3) **A comparison with published mechanisms.** At a fixed step count and skip schedule, Trajector outperforms reproductions of TaylorSeer and Block Caching on SSIM, PSNR, LPIPS and ImageReward.

2. Related Work

Faster solvers [9], [10] and distillation [11] shorten the sampling trajectory; caching makes each remaining step cheaper. We hold the step count and solver fixed, which isolates caching but does not rank the two routes.

2.1. Training-Free Feature Caching

DeepCache [2] reuses U-Net features; FORA [3] reuses transformer block residuals on a fixed interval and Δ -DiT [4]

caches different blocks early and late. TeaCache [5], ToCa [12], FasterCache [13] and AdaCache [6] adapt *when* to skip, then reuse verbatim. We take a uniform schedule as given and ask what to add back.

2.2. Learned Corrections and Training Objectives

Learning-to-Cache [14] learns which layers may be skipped. LazyDiT [15] learns from each block’s input when the block can be skipped and its cached output reused. Block Caching [8] learns a per-channel scale and shift for each timestep: trained, but a function of the timestep alone. TaylorSeer [7] replaces verbatim reuse with a forecast of the cached residual from finite differences, with no training. Unlike all four, our corrector uses the *current state* to correct the reused residual. Its nonzero Jacobian makes errors compound across injection sites (Sec. 5.2), which only a trajectory-level loss sees. HarmoniCa [16] already names an “objective mismatch” and fixes it by training along the denoising process with a proxy for final image error; we do not claim the mismatch as novel. We add a more local diagnosis (residual error is pointwise, cache error integrates) and a fix with no image-space proxy: the cached rollout is differentiable, so trajectory deviation can be the loss. P2 [17] and Min-SNR [18] reweight the residual target; they do not replace it.

3. The Residual Objective

The measurements below use PixArt- Σ -XL-2-512 [19] (0.6B parameters, 28 blocks) with its shipped DPM-Solver++ scheduler [10] at 20 steps and 512². Anchors are uniform at interval $i=5$ (4 anchor steps, 16 reuse steps). All image and trajectory numbers are against the exact 20-step output from the same seed, on the 24 test prompts of Sec. 5. The 28 blocks are grouped into K contiguous cached segments, so K is the number of injection sites per reuse step: $K=1$ corrects the whole stack once and $K=28$ corrects every block.

3.1. Trajectory Error

At each of the 20 steps we measure $\|x_t - x_t^*\|/\|x_t^*\|$, where x_t is the cached latent after step t and x_t^* the exact one. Averaged over the 24 prompts, deviation rises from 0.0089 to 0.5856. It dips once, at the second anchor (step 5); later anchors only slow its growth. This is the basic reason the residual target is misaligned: the corrector’s action at step t changes the state at $t+1$, while (1) is defined on the unperturbed trajectory and residual regression (Sec. 4) never differentiates through the reuse step.

3.2. Residual Retraining

To check that residual regression had not simply been trained too little, we gave every K the same 50,176 training examples, one per (image, reuse step, site), by sampling

28/ K seeds per prompt. We subsampled 48 of 1024 tokens per site and trained each K for the step budget at which a doubling search found the next doubling to improve residual error by under 0.5%. Residual correctors keep the checkpoint with the lowest residual error on validation-prompt features, where residual errors are also measured. Against correctors trained with one seed per prompt and a 16k-step budget, residual error fell 25–30% for $K \leq 7$, while test SSIM changed by +0.012, +0.012, −0.002 and +0.006 at $K=1, 2, 4, 7$ ($t=2.9, 2.7, -0.5, 2.0$), against +0.039 for adding correction at all ($K=1$). Residual accuracy is a poor guide to what a corrector is worth.

3.3. Oracle Correction

An oracle that recovers a fraction β of the true residual has residual error $(1 - \beta)^2$. Final-step trajectory error falls from 0.586 at $\beta=0$ to 0.381 at $\beta=0.8$, and to 0 at $\beta=1$. The curve falls slowly up to $\beta=0.8$ and steeply after it. Additive injection can in principle recover the exact output ($\beta=1$); partial residual accuracy simply buys little. Each residual-trained corrector lowers final error 0.9–1.1 $\times$ as much as an oracle with its residual error, but a better residual fit barely moves it: the 27% residual gain of matched-data retraining at $K=1$ maps to $0.515 \rightarrow 0.492$ on the curve, yet the real correctors move $0.505 \rightarrow 0.502$. Trajectory reaches 0.462, close to an oracle that recovers 60% of the residual (0.457).

4. Method

4.1. Step Caching

A DiT block b is residual, $R_b(h) = \text{block}_b(h) - h$, where h is its input hidden state (1024 tokens $\times$ 1152 channels), so one exact block maps $h \mapsto h + R_b(h)$. Let $t = 0, \dots, T-1$ index solver steps ($T=20$) and $\mathcal{A}$ be the anchor set (here $\{0, 5, 10, 15\}$ at interval $i=5$). On $t \in \mathcal{A}$ every block runs and the residual is stored. On a reuse step the stored residual is added instead:

$$h \mapsto h + R_b(h_a), \quad a = \max\{s \in \mathcal{A} : s \leq t\}, \quad (2)$$

where a is the latest anchor and h_a the input the block saw there. With blocks grouped into K sites, the reuse error at site k of step t is $\Delta R_{t,k} = R_{t,k}^{\text{true}} - R_{t,k}(h_a)$, i.e. (1). The residual objective regresses the corrector’s prediction $\widehat{\Delta R}_{t,k}$ onto it on the *unperturbed* hidden states of the exact trajectory,

$$\mathcal{L}_{\text{res}} = \sum_{t \notin \mathcal{A}} \sum_{k=1}^K \|\Delta R_{t,k} - \widehat{\Delta R}_{t,k}\|^2, \quad (3)$$

a mean squared error with targets divided by their global RMS. No derivative is taken through (2), so (3) cannot see that a correction at step t changes the state at $t+1$.

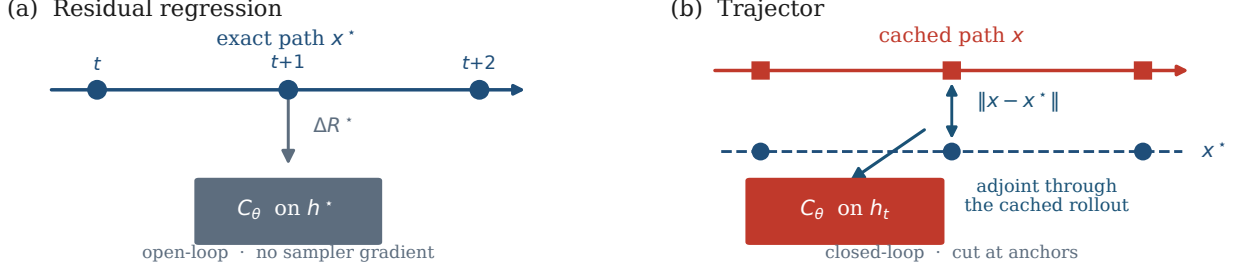


Figure 1. (a) Residual regression supervises C_θ on the exact path. (b) Trajectory trains C_θ by the discrete adjoint of the cached rollout. Architecture and the skip schedule are in the text.

4.2. Corrector

The 28 blocks are partitioned into K contiguous cached segments. A shared network C_θ is applied at each site. At a site it reads the state drift $\Delta h = h_t - h_a$, the residual R_a the site’s blocks produced at the anchor, and the anchor hidden state h_a , conditioned on the step t (through its diffusion timestep), the reuse horizon $\tau = t - a$ and the site index k . Each input is divided by its RMS over the training set ($s_{\Delta h}$, s_R , s_h) and the head is rescaled by the target RMS $s_{\Delta R}$,

$$\widehat{\Delta R} = s_{\Delta R} C_\theta \left(\frac{\Delta h}{s_{\Delta h}}, \frac{R_a}{s_R}, \frac{h_a}{s_h} \mid t, \tau, k \right), \quad (4)$$

without which the three inputs differ in scale by up to about $180\times$. A conditioning vector $c = \text{MLP}([\gamma(t); \gamma(\tau); e_k])$ (Fourier features γ , site embedding e_k) modulates, through AdaLN, a trunk of four residual MLP blocks of width 512, one 4-head attention layer across tokens, and a rank-256 linear path per site. The output projection is zero-initialised, so $C_\theta=0$ is verbatim reuse (11.9M parameters at $K=1$, 43.7M at $K=28$). The site’s output is

$$h_{\text{out}} = h_t + R_a + \sigma \widehat{\Delta R}, \quad (5)$$

with injection strength σ selected on validation prompts, never on test prompts. The Jacobian $J_k = \partial \widehat{\Delta R}_k / \partial h$ is nonzero because C_θ reads Δh ; that is what turns error across sites multiplicative (Sec. 5.2).

4.3. Pathwise Adjoint Training

Trajectory keeps (4)–(5) and replaces (3). Let x_t be the latent after step t of the cached rollout under (5) and x_t^* the exact latent under the same prompt and seed. The pathwise objective is

$$\mathcal{L} = \sum_{t \notin \mathcal{A}} \frac{\|x_t - x_t^*\|^2}{\|x_t^*\|^2}, \quad (6)$$

minimised by reverse-mode differentiation through the cached rollout, which is the discrete adjoint of the sampler restricted to reuse steps (Fig. 1b). No labels are needed: x^* is the exact rollout for the same prompt and seed, computed once without gradients.

The adjoint is cheap for a reason specific to caching. A cached block never runs block_b; the runtime returns (5), so

the graph over one reuse step is K applications of C_θ , not the 28 blocks of a 0.6B DiT. Peak memory is 7.5 GB at $K=1$ and 12.8 GB at $K=2$; checkpointing each corrector call keeps $K \geq 4$ at 3–6 GB. Gradients are cut at anchors, where every block runs again on the current state, so a corrector call influences later intervals only through the latent.

Trajectory is a second training stage, not a replacement trained from scratch: residual regression is a cheap, teacher-forced way to reach a corrector that already helps, just the wrong *final* objective. We initialise from the matched-data residual corrector of the same K (Sec. 3.2), which is also the “Residual obj.” arm below, so each comparison isolates the second stage; training from zero initialisation is untested. Training uses AdamW, learning rate 10^{-5} , gradients clipped to norm 1.0, 400 steps, one prompt per step, with σ fixed during training (0.75 at $K=1$, 0.3 at $K=2, 4$, 0.2 above). We keep the checkpoint with the lowest trajectory error on 12 training prompts held back from the 48 used for updates. C_θ is trained in fp32 against an fp16 transformer; at inference it runs in fp16 at $K=1$ and fp32 at $K \geq 2$. Measured cost: 2.57 TFLOP per anchor step, 0.041 TFLOP per corrector call, 2.66 TFLOP fixed per image. One run takes 23 minutes at $K=1$ on an A40 shared with other jobs, and 43–90 minutes with checkpointing at $K \geq 4$.

5. Experiments

Prompts are templated from 12 styles, 39 subjects and 12 conditions, and split by subject into 112 training, 24 validation and 24 test prompts (27, 6 and 6 subjects). Every corrector’s σ is selected on validation $\times 3$ seeds over one grid of 13 values shared by all correctors. Every reported image metric is a paired comparison on the test prompts $\times$ the same 3 seeds ($n=72$), which selection never sees. We report SSIM [20], PSNR, LPIPS [21] and ImageReward [22], with paired t statistics and the number of improved cases.

5.1. Comparison at a Fixed Step Count

Table 1 compares at 20 steps and $i=5$, so every arm skips the same blocks and speedup differs only by each method’s correction overhead. We reproduce the TaylorSeer mechanism [7] (a forecast of the residual from finite differences of its

TABLE 1. FIXED 20 STEPS, $i=5$, TEST PROMPTS, $n=72$. “Sp.” IS MEDIAN WALL-CLOCK SPEEDUP AGAINST THE EXACT MODEL IN THE SAME RUN, IR IS IMAGEReward [22], Δ SSIM IS PAIRED AGAINST VERBATIM CACHING, AND BOLD MARKS THE BEST ACCELERATED METHOD.

Method	Sp.	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	IR $\uparrow$	Δ SSIM
Exact, 20 steps	1.00	—	—	—	+1.43	—
Verbatim cache	3.75	0.4635	14.29	0.5366	+0.70	—
TaylorSeer 1st	3.67	0.4341	13.02	0.5544	+0.32	−0.0294
TaylorSeer 2nd	3.65	0.3546	12.18	0.6252	−0.56	−0.1089
Block Caching	3.51	0.4788	14.64	0.5411	+0.64	+0.0153
Residual obj.	3.40	0.5145	15.12	0.4370	+1.17	+0.0511
Trajector	3.39	0.5320	15.27	0.4423	+1.05	+0.0686

own history, with no training) and the Block Caching mechanism [8] (a per-channel scale and shift for each timestep, trained but blind to the state). Both are reimplemented in our runtime rather than run from the authors’ code, so they are reproductions of the mechanisms under our schedule and backbone, not replications of the published results.

Paired against Trajector, the first-order TaylorSeer mechanism loses 0.0979 SSIM ($t=12.8$, 67/72) and 0.1121 LPIPS ($t=15.3$); the second-order version loses 0.1774 ($t=22.0$, 72/72); Block Caching loses 0.0532 ($t=9.5$, 64/72) and 0.0988 LPIPS ($t=12.9$). Correction trained on residuals already outperforms both reproduced mechanisms. Trajector adds +0.0175 SSIM over it ($t=4.5$), but at $K=1$ no significant PSNR or LPIPS gain (+0.15 dB, $t=1.2$; 0.005 worse LPIPS, $t=-1.1$).

Under ImageReward, first-order TaylorSeer scores below verbatim caching (-0.38 , $t=-4.8$) and Block Caching does not improve on it (-0.06 , $t=-1.0$). Only the two state-dependent learned correctors do (+0.47 and +0.35, $t=6.2$ and 4.5), and the residual objective scores higher than Trajector (+0.12, $t=2.3$): at $K=1$, Trajector’s advantage is fidelity, not preference.

TaylorSeer at $i=5$ is worse than no correction (-0.0294 , $t=-8.3$); in an earlier run of our reproduction it helped at $i=2-3$ (+0.008 and +0.012 SSIM, $n=192$), so this is not a claim against its own operating point. It also needs no training; Trajector’s 11.9M parameters and 0.041 TFLOP per correction cost 7.6% of speedup relative to it and 9.7% relative to verbatim caching.

5.2. Injection Granularity

Table 2 and Fig. 2 show Trajector ahead of the residual objective at all six granularities, every difference significant. The margin is smallest at $K=1$ (+0.0175) and two to three times larger at $K\geq 4$ (+0.037 to +0.047), where the residual objective’s own gain has fallen to +0.006–0.020. This is consistent with the mechanism behind the degradation. A correction that reads the state makes error propagate across sites multiplicatively rather than additively, $\delta_{k+1} = (I + \sigma J_k)\delta_k + \varepsilon_k$, where δ_k is the deviation of the hidden state from the exact path on entering site k , ε_k is the residual error committed there, and $J_k = \partial \hat{\Delta} R_k / \partial h$ comes from

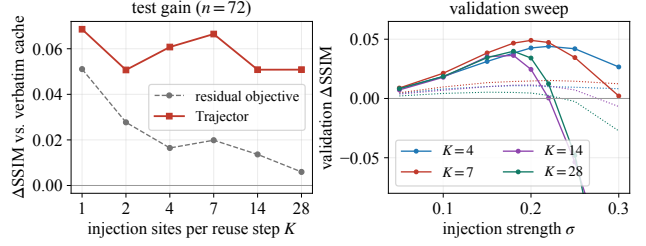


Figure 2. Left: test gain over verbatim caching at each granularity ($n=72$), each corrector at its validation-selected σ . Right: validation sensitivity to σ for $K\geq 4$ (solid: Trajector, dotted: residual objective).

TABLE 2. TEST GAIN IN SSIM OVER VERBATIM CACHING, $n=72$, EACH CORRECTOR AT ITS VALIDATION-SELECTED σ .

K	Residual obj.	Trajector	Difference
1	+0.0511 ($\sigma=0.6$)	+0.0686 ($\sigma=0.6$)	+0.0175 ($t=4.5$)
2	+0.0277 ($\sigma=0.3$)	+0.0507 ($\sigma=0.25$)	+0.0230 ($t=4.6$)
4	+0.0165 ($\sigma=0.18$)	+0.0608 ($\sigma=0.22$)	+0.0443 ($t=10.0$)
7	+0.0199 ($\sigma=0.22$)	+0.0665 ($\sigma=0.2$)	+0.0466 ($t=7.4$)
14	+0.0136 ($\sigma=0.2$)	+0.0508 ($\sigma=0.18$)	+0.0372 ($t=5.6$)
28	+0.0060 ($\sigma=0.15$)	+0.0508 ($\sigma=0.18$)	+0.0449 ($t=8.7$)

(4). Only Trajector observes the result of that compounding; residual training sees pointwise error. From $K=1$ to $K\geq 7$ the residual objective’s gain falls 2.6–8.6 $\times$, Trajector’s by at most 1.35 $\times$. Trajector also improves LPIPS at every $K\geq 2$ ($t=2.7$ to 9.9), though not at $K=1$.

Fig. 2 (right) also shows why σ must be selected per corrector. Under both objectives the validation optimum moves from 0.6 at $K=1$ to 0.15–0.22 at $K\geq 4$, so this is a property of granularity rather than of the objective. Trajector’s peak also narrows: at $K=14$ and 28 it falls from its maximum to zero or below within 0.04–0.07 of σ .

In Fig. 3, verbatim caching smears fine structure, TaylorSeer adds chromatic fringing (worst LPIPS in Table 1) and Block Caching leaves colour blotches. Trajector keeps the most structure, such as the typewriter keys and the window tracery, though blocky artefacts remain.

5.3. Scope and Limitations

We hold the step count fixed because on this backbone ImageReward saturates by 10 solver steps, so an equal-compute comparison mostly measures solver convergence. Results are on 512² PixArt- Σ with templated prompts; the 72 test cases come from 6 subjects. With seeds averaged per prompt ($n=24$), every Trajector-over-residual SSIM gain keeps $t>3.1$, while the ImageReward difference and the frozen-schedule lead below fall to $t\approx 2.1$. Averaged per subject ($n=6$, the unit of the split), Trajector still leads at every K , in 4–6 of the 6 subjects ($t=2.6$ to 6.2).

Schedules. Trajector is trained with uniform anchors, but C_θ is conditioned on τ rather than i , and (6) holds for any anchor set. Adaptive policies (TeaCache, AdaCache) read the corrected state, so for a controlled test we froze, per

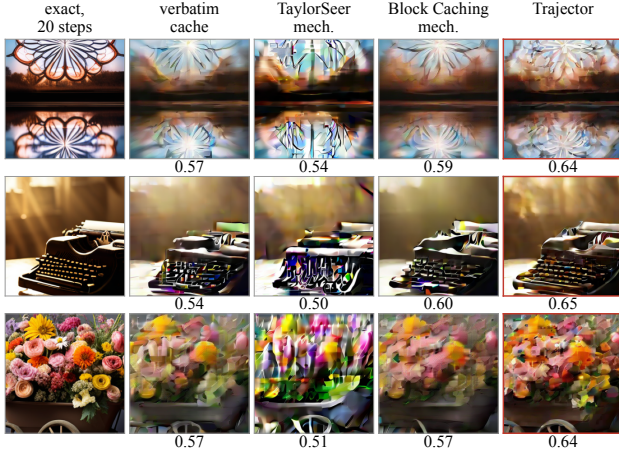


Figure 3. 250^2 crops at native resolution, same seed and prompt across a row, full-image SSIM against the exact output below each panel. Each row is the case with the median Trajector-over-cache SSIM gain for one of the first three test subjects, not a case picked by eye.

TABLE 3. THE $K=1$ CORRECTORS, TRAINED WITH UNIFORM ANCHORS, UNDER UNIFORM ANCHORS AND UNDER THE FROZEN TEACACHE-STYLE SCHEDULE (TEST, $n=72$).

Method	Uniform anchors		Frozen schedule	
	SSIM $\uparrow$	LPIPS $\downarrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Verbatim cache	0.4635	0.5366	0.4647	0.6031
Residual obj.	0.5145	0.4370	0.4846	0.5438
Trajector	0.5320	0.4423	0.4916	0.5199

test case, the schedule a TeaCache-style rule (accumulated relative change, without its fitted rescaling) picks on an uncorrected rollout, and replayed it for every arm. It refreshes 4.0 times on average, like the uniform schedule, but unevenly (e.g. anchors $\{0, 11, 15, 18\}$), so τ reaches 8–11 against 4 in training. Table 3 gives the result. Without retraining, both $K=1$ correctors still help (+0.020 and +0.027 SSIM, $t=4.1$ and 5.1) and Trajector still leads (+0.007, $t=2.4$), but the gains shrink to about 40% of their uniform-schedule values. The schedule hurts verbatim caching perceptually (LPIPS $0.537 \rightarrow 0.603$) while leaving its SSIM unchanged, and on it Trajector’s lead extends to LPIPS (0.024 better, $t=5.7$) and PSNR (+0.28 dB, $t=4.4$), neither of which it has under uniform anchors at $K=1$. In LPIPS the residual objective loses more of its gain over verbatim caching ($0.100 \rightarrow 0.059$) than Trajector does ($0.094 \rightarrow 0.083$). Training on the deployed schedule is the natural remedy and is untested.

Gains come from 400 optimiser steps without hyperparameter search, so they are not an upper bound. Correctors at $K \geq 2$ run in fp32 to avoid fp16 overflow.

6. Conclusion

Regressing the residual a cache corrector should have produced is misaligned with what caching damages: cache error integrates along the trajectory, residual error is point-

wise. Fine-tuning the same corrector against trajectory deviation improves SSIM at every injection granularity we tested and LPIPS at every $K \geq 2$. At a fixed step count it outperforms reproductions of two published mechanisms on all four metrics. Under the preference model, state-dependent learned correction is the only approach that improves on verbatim caching, with the residual objective ahead of Trajector at $K=1$. Code, configurations, logs and per-case metrics are at <https://github.com/xiaruize0911/Diff-cache-Comp>; dit-residual-delta/scripts/cr_report.py recomputes every table cell from the archived results.json files.

References

- [1] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *ICCV*, 2023.
- [2] X. Ma, G. Fang, and X. Wang, “DeepCache: Accelerating diffusion models for free,” in *CVPR*, 2024.
- [3] P. Selvaraju, T. Ding, T. Chen *et al.*, “FORA: Fast-forward caching in diffusion transformer acceleration,” *arXiv:2407.01425*, 2024.
- [4] P. Chen, M. Shen, P. Ye *et al.*, “ Δ -DiT: A training-free acceleration method tailored for diffusion transformers,” *arXiv:2406.01125*, 2024.
- [5] F. Liu, S. Zhang, X. Wang *et al.*, “Timestep embedding tells: It’s time to cache for video diffusion model,” in *CVPR*, 2025.
- [6] K. Kahatapitiya, H. Liu, S. He *et al.*, “Adaptive caching for faster video generation with diffusion transformers,” in *ICCV*, 2025.
- [7] J. Liu, C. Zou, Y. Lyu *et al.*, “From reusing to forecasting: Accelerating diffusion models with TaylorSeers,” in *ICCV*, 2025, arXiv:2503.06923.
- [8] F. Wimbauer, B. Wu, E. Schoenfeld *et al.*, “Cache me if you can: Accelerating diffusion models through block caching,” in *CVPR*, 2024.
- [9] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *ICLR*, 2021.
- [10] C. Lu, Y. Zhou, F. Bao *et al.*, “DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models,” *Machine Intelligence Research*, vol. 22, pp. 730–751, 2025.
- [11] Y. Song, P. Dhariwal, M. Chen *et al.*, “Consistency models,” in *ICML*, 2023.
- [12] C. Zou, X. Liu, T. Liu *et al.*, “Accelerating diffusion transformers with token-wise feature caching,” in *ICLR*, 2025.
- [13] Z. Lv, C. Si, J. Song *et al.*, “FasterCache: Training-free video diffusion model acceleration with high quality,” in *ICLR*, 2025.
- [14] X. Ma, G. Fang, M. B. Mi *et al.*, “Learning-to-Cache: Accelerating diffusion transformer via layer caching,” in *NeurIPS*, 2024.
- [15] X. Shen, Z. Song, Y. Zhou *et al.*, “LazyDiT: Lazy learning for the acceleration of diffusion transformers,” in *AAAI*, 2025.
- [16] Y. Huang, Z. Wang, R. Gong *et al.*, “HarmoniCa: Harmonizing training and inference for better feature caching in diffusion transformer acceleration,” in *ICML*, 2025, arXiv:2410.01723.
- [17] J. Choi, J. Lee, C. Shin *et al.*, “Perception prioritized training of diffusion models,” in *CVPR*, 2022.
- [18] T. Hang, S. Gu, C. Li *et al.*, “Efficient diffusion training via Min-SNR weighting strategy,” in *ICCV*, 2023.
- [19] J. Chen, C. Ge, E. Xie *et al.*, “PixArt- Σ : Weak-to-strong training of diffusion transformer for 4K text-to-image generation,” in *ECCV*, 2024.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh *et al.*, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros *et al.*, “The unreasonable effectiveness of deep features as a perceptual metric,” in *CVPR*, 2018.
- [22] J. Xu, X. Liu, Y. Wu *et al.*, “ImageReward: Learning and evaluating human preferences for text-to-image generation,” in *NeurIPS*, 2023.